"Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos"

A

Project Report

submitted

in partial fulfillment

for the award of the Degree of

Bachelor of Technology

in Department of Computer Science and Engineering(DS)

Project Id:
SKIT/2025-26/DS/024

Project Mentor:
Mr. Kailash Soni
(Associate Professor)

Submitted By:
Prachi Shrimali (22ESKCX085)
Manuraj Singh (22ESKCX067)
Piyush Kewlani (22ESKCX084)
Mitesh Jangid (22ESKCX070)

Department of Computer Science and Engineering
Swami Keshvanand Institute of Technology, M & G, Jaipur
Rajasthan Technical University, Kota
Session 2025-2026

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that **Ms Prachi Shrimali**, a student of B.Tech(Computer Science & Engineering(DS)) VIIth semester has submitted her Project Report entitled **"Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos"** under my guidance

Mentor	**Coordinator**
Mr Kailash Soni	Mr Sumit Kumar
Associate Professor	Assistant Professor
Signature.....	Signature.....

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that **Mr Manuraj Singh**, a student of B.Tech(Computer Science & Engineering(DS)) VIIth semester has submitted her Project Report entitled **"Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos"** under my guidance

Mentor	**Coordinator**
Mr Kailash Soni	Mr Sumit Kumar
Associate Professor	Assistant Professor
Signature.....	Signature.....

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that **Mr Piyush Kewlani**, a student of B.Tech(Computer Science & Engineering(DS)) VIIth semester has submitted her Project Report entitled **"Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos"** under my guidance

Mentor	**Coordinator**
Mr Kailash Soni	Mr Sumit Kumar
Associate Professor	Assistant Professor
Signature.....	Signature.....

Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that **Mr Mitesh Jangid**, a student of B.Tech(Computer Science & Engineering(DS)) VIIth semester has submitted her Project Report entitled **"Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos"** under my guidance

Mentor	**Coordinator**
Mr Kailash Soni	Mr Sumit Kumar
Associate Professor	Assistant Professor
Signature.....	Signature.....

DECLARATION

We hereby declare that the report of the project entitled **"Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos"** is a record of an original work done by us at Swami Keshvanand Institute of Technology, Management and Gramothan, Jaipur under the mentorship of "Mr. Kailash Soni"(Dept. of Computer Science and Engineering) and coordination of "Mr. Sumit Kumar" (Dept.of Computer Science and Engineering). This project report has been submitted as the proof of original work for the partial fulfillment of the requirement for the award of the degree of Bachelor of Technology (B.Tech) in the Department of Computer Science & Engineering(DS).It has not been submitted anywhere else, under any other program to the best of our knowledge and belief.

Team Members **Signature**

Prachi Shrimali (22ESKCX085)

Manuraj Singh (22ESKCX067)

Piyush Kewlani (22ESKCX084)

Mitesh Jangid (22ESKCX070)

Acknowledgement

A project of such a vast coverage cannot be realized without help from numerous sources and people in the organization.We take this opportunity to express our gratitude to all those who have been helping us in making this project successful.

We are highly indebted to our mentor **Mr. Kailash Soni**. He has been a guide, motivator source of inspiration for us to carry out the necessary proceedings for the project to be completed successfully. We also thank our project coordinator **Mr. Sumit Kumar** for his co-operation, encouragement, valuable suggestions and critical remarks that galvanized our efforts in the right direction.

We would also like to convey our sincere thanks to **Dr. Mehul Mahrishi**, HOD, Department of Computer Science and Engineering, for facilitating, motivating and supporting us during each phase of development of the project.Also, we pay our sincere gratitude to all the Faculty Members of Swami Keshvanand Institute of Technology, Management and Gramothan, Jaipur and all our Colleagues for their co-operation and support.

Last but not least we would like to thank all those who have directly or indirectly helped and cooperated in accomplishing this project.

Team Members:

Prachi Shrimali (22ESKCX085)

Manuraj Singh (22ESKCX067)

Piyush Kewlani (22ESKCX084)

Mitesh Jangid (22ESKCX070)

Abstract

The project "Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos" focuses on automating the extraction and validation of information from images collected in real-world environments. In many field-based applications such as surveys, inspections, and data collection, manual verification of images is time-consuming, error-prone, and inefficient. This project aims to solve this problem by leveraging Artificial Intelligence and Computer Vision techniques.

Here's what actually happens: you drop in a bunch of field photos and let the system do its thing. It statrs by cleaning up the data, then hunts for important objects in each image with a YOLO-based model. Got text in the photo? Optical Character Recognition (using Tesseract) grabs that, too. After pulling out all the details, the system double-checks everything against a rulebook to make sure it's accurate.

To keep the AI sharp and dependable, we fine-tune it with techniques like data augmntation and hyperparameter tuning. You don't need to geek out over code to use it—the interface (built with Streamlit or Tkinter) is straightforward. Upload your images, check out the highlights and extracted text, and export all your results in a neat CSV or text file.

At the end of the day, this project shows that AI is up for the job when it comes to tough, repetitive field data work. It handles images in a flash, slashes errors, and can scale up for bigger tasks without breaking a sweat. Instead of bogging yourself down with manual checks, you get a smart, efficient system that validates your crowd-sourced field photos in real time.

Table of Content

1 Introduction **2**

1.1 Problem Statement and Objective 2

1.2 Literature Survey /Market Survey/Investigation and Analysis 3

1.3 Introduction to Project 4

1.4 Proposed Logic / Algorithm / Business Plan / Solution 4

1.5 Scope of the Project 5

2 Software Requirement Specification **7**

2.1 Overall Description 7

2.1.1 Product Perspective 7

2.1.1.1 System Interfaces 7

2.1.1.2 User Interfaces 8

2.1.1.3 Hardware Interfaces 9

2.1.1.4 Software Interfaces 9

2.1.1.5 Functional Requirement 10

2.1.1.6 User Characteristics 10

2.1.1.7 Constraints 11

2.1.1.8 Assumption and Dependencies 11

3 System Design Specification **12**

3.1 System Architecture 12

3.2 Module Decomposition Description 13

3.3 High Level Design Diagrams 13

3.3.1 Use Case Diagram 14

3.3.2 Data-Flow Diagram 15

3.3.3 Class Diagram 17

3.3.4 Sequence Diagram 18

4 Methodology and Team **20**
4.1 Introduction to Agile Methodology 20
4.2 Team Members, Roles & Responsibilities 22

5 System Testing **26**
5.1 Introduction 26
5.2 Types of Testing Performed 26
5.2.1 Unit Testing 27
5.2.2 Integration Testing 28
5.2.3 System Testing 29
5.3 Test Cases 30

6 Project Screen Shots **34**

7 Conclusion and Future Scope **38**
7.1 Conclusion 38
7.2 Future Scope 38

8 UN Sustainable Development Goals **40**
8.1 Mapping with UN sustainable development goals 40

Project Poster **43**

Certificate **44**

Plag Report **45**

GitHub Link **46**

List of Figures

3.1 Use Case diagram 14
3.2 DFD-0 15
3.3 DFD-1 16
3.4 Class Diagram 17
3.5 Sequence Diagram 18

4.1 Agile model 21

6.1 Home Page 34
6.2 Image Upload Interface 35
6.3 System Dashboard and Analytics View 36
6.4 Result Output and Export Data View 37

8.1 Final Project Poster 43
8.2 Acceptance Email for Research Paper 44

List of Tables

4.1 Member-1 Roles and Responsibilities 22
4.2 Member-2 Roles and Responsibilities 23
4.3 Member-3 Roles and Responsibilities 24
4.4 Member-4 Roles and Responsibilities 25

5.1 Test Case table for Image Input and Preprocessing Module 30
5.2 Test Case table for Object Detection Module 31
5.3 Test Case table for OCR Module 32
5.4 Test Case table for Validation Module 32
5.5 Test Case table for Integration Module 33
5.6 Test Case table for UI Module 33

8.1 SDG Mapping 41

Chapter 1
Introduction

1.1 Problem Statement and Objective

These days, people collect a ton of field data using images—for inspection, monitoring and verification. The problem is, everything comes from different people, on different devices, under all kinds of lighting, angles, and backgrounds. So, the images end up all over the place. That makes the data pretty inconsistent and sometimes not even that reliable. Usually, someone has to sit down and check everything by hand. It's slow, tedious work, and it only gets worse if there are lots of images to go through. It's easy to make mistakes, too—missed details, misread labels or IDs, or just plain overlooking stuff.

There's another snag. Right now, there isn't really a solid system that can find objects in images, pull out any text—like codes or labels—and then actually check if that info is right So, things feel clunky and disorganized, especially when you've got a huge pile of data to sort through. We really need a smarter way to handle this, something that can process the images, find what matters, grab any important text, and check that it all makes sense—without making people do everything themselves.

That's where this project comes in. The aim is simple: build an automated system using AI to get this job done. You feed the images in, and the system handles it from there—preprocesing, spotting objects, grabbing any text it finds, and then checking everything against a set of rules. By bringing in models like YOLO for object detection and Tesseract for OCR, we can boost both accuracy and speed. There's also a straightforwrd interface so users can upload their images, see results, and even export the findings if they need to. In the end, the goal is to cut down on manual work, cut down on errors, and just make the whole thing run faster and smoother.

1.2 Literature Survey /Market Survey/Investigation and Analysis

Before starting this project, I took a good look at the tools and techniques people use for image-based data extraction and validation. Computer vision and OCR? They're all over the place — from document scanning to license plate recognition, invoices, field inspections, you name it. Models like YOLO stand out for real-time object detection, mostly because they're fast and accurate. Tesseract is everyone's go-to for pulling text from images; it's open-source, easy to drop into different setups, and just gets the job done.

But here's what struck me: most solutions out there stick to either object detection or text extraction — rarely both. You don't see many systems that pull these togther, especially when dealing with field data from a bunch of people. And let's be real, crowd-sourced images vary wildly in quality. That makes detection and extraction kind of a headache. Another thing missing? Proper validation. Lots of tools can grab text, but hardly any check if that text is correct or fits the format it's supposed to.

Industries like agriculture, infrastructure, logistics, and all those survey-based organizations are leaning heavily on image-based data collection now. They need fast, reliable systems to churn through stacks of images and actually make sense of the data. Relying on manual checks eats time and money, so automated solutions aren't just nice-to-have — they're in demand. People want tools that analyze images and confirm the extracted info is legit.

So after diving into all this, it's pretty clear: there's a real need for a system that fuses object detection, OCR, and validation into one streamlined flow. That's the whole point of this project — to build a single, efficient pipeline that handles preprocesing, finds features, extracts text, and validates everything in a way that's both reliable and easy to use.

1.3 Introduction to Project

The project, "Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos," is all about building a smart system that can look at photos from users and pull out useful information automatically. In lots of real-life situations, people rely on images as a data source, but going through them by hand just takes too long. This project tackles that by using computer vision and OCR to speed everything up.

Here's how it works. The system takes an uploaded image and cleans it up first—sharpening things, boosting the quality—so it's ready to be analyzed. Then, it runs the image through a YOLO-based object detector to hunt down the important bits or features in the photo. Once those regions are found, OCR kicks in and grabs any text inside them. After that, the system checks the extracted text to make sure it matches the right format and rules.

There's also a simple interface where you can drop in your images and instantly see what the system finds. It highlights detected objects with boxes and shows the pulled-out text, so everything's clear at a glance. You can even export your results if you want to use them elsewhere. By connecting all these technologies in one pipeline, the project makes field data analysis a whole lot quicker, more reliable, and way less work for people.

1.4 Proposed Logic / Algorithm / Business Plan / Solution

Here's how the system works, start to finish. Someone uploads a field image through a simple interface. Instead of breaking the process into separate steps—like detection, text extraction, and validation—the system pulls them all together into one automated pipeline. That way, everything happens quickly and you don't have to double-check or repeat tasks.

Right after upload, the image goes through a quick clean-up phase. The system resizes, normalizes, and removes any noise—basic stuff to make sure the models have the best shot at reading the image clearly. Once that's done, it sends the prepped

image to a YOLO-based object detection model. This model hunts for the important areas—maybe numbers, codes, or text fields—and draws boxes around them.

Next up, it passes those marked regions to Tesseract OCR. That's the tool in charge of grabbing the text out of each box. The system knows OCR is rarely perfect, especially with real-world images, so it doesn't stop there. It scrubs and polishes the output, cleaning up typos or formatting issues. Then, it runs a set of validation rules tailored to the expected data form—like pattern checks, character limits, or verifying values—just to make sure nothing slips through the cracks.

The pipeline script keeps all these pieces moving in sync, shuffling data from one step to the next. At the end, results pop up in the interface. Users see exactly which objects the system found, along with the cleaned, validated text from each. For convenience, there's also a button to export everything as a CSV or TXT file.

In short, this solution takes the busywork out of image processing. It handles every step automatically and checks the data before it lands on your desk—so you can trust the output without pouring over details by hand.

1.5 Scope of the Project

The scope of this project mainly focuses on developing an automated system that can analyze field images and extract useful information from them in an efficient way It handles photos from many users, running them through object detection and OCR to spot key features, grab text from the right spots, and check the data against set rules. So, if you're working with piles of images that need to be sorted and verified, this system steps up.

There's an easy-to-use interface, too. You just upload your images, see what the system found, and export the results if you need to. Everything's laid out clearly, with detected objects and text right there for you. It suits jobs like inspections, surveys, or documentation—basically anywhere speed and accuracy are a must.

At the moment, it's best with images that aren't too complicated and stick to known patterns and text styles. Results can depend on how sharp the image is, how good the lighting was, and how clear the text shows up. In the future, the plan is

to make the models smarter, handle trickier images, and add stronger validation to boost the system's reliability and scale.

Chapter 2
Software Requirement Specification

2.1 Overall Description

This system takes field images and pulls out all the key info you need, automatically checking to make sure it's accurate. Everything—image procesing, object detection, and text extraction—happens in one smooth workflow. All you do is upload your images, and the system does the rest, moving them through several stages and giving you useful, verified results.

It's built for situations where you've got tons of pictures coming in from different places, and sorting through them by hand would just take forever. Instead this system does the job for you. It starts by prepping each image, then uses a trained model to spot important objects, runs OCR to read any text, and finally applies validation rules to make sure the data checks out.

Each part of the process—preprocesing, detection, OCR, and validation—is its own module. They work on their own but connect through one main pipeline, so everything flows together smoothly There's a user interface, too, making it simple to upload images, see the results, and export whatever you need. In short, this application gives you an easy, efficient, and reliable way to handle large sets of field images, all thanks to AI-driven tools.

2.1.1 Product Perspective

2.1.1.1 System Interfaces

The system is built so each part talks to the others and keeps everything running smoothly. Here, everything works like a pipeline—one module hands off to the next in a clear order. It all starts when the user uploads an image using the interface. That image heads straight to the backend, where a series of processing steps kick in, one

after another.

First up, the preprocesing module takes the image and cleans it up. Things like resizing, normalization, and noise reduction happen here to improve the image quality. After that, the object detection module comes in. Using the YOLO model, it spots important features and highlights them with bounding boxes. Only the key regions matter, so the system extracts those and sends them off to the OCR module to recognize any text.

The next stop is validation. The system checks the extracted text against a set of rules and formats to make sure it's accurate and fits what's expected.

When all that's done, the results—both the detected objects and the validated text—go back to the frontend so the user can see them. Data keeps moving smoothly between frontend and backend, which keeps everything efficient and the results reliable. This setup, with clear handoffs and separate modules, not only speeds up debugging and maintenance but also makes it simple to upgrade any part of the system on its own.

2.1.1.2 User Interfaces

The user interface plays an important role in making the system easy to use and accessible to different types of users. In this project, the interface is designed with simplicity and clarity in mind so that users can interact with thesystem without facing any difficulty. The main goal is to let people upload images, check out what the system finds, and actually understand what's going on.

Once someone uploads an image, the interface shows what's happening—it lets you know the image is processing. When that's done, you get a clean display: detected objects show up with bounding boxes, and any text the system finds is front and center. All of this makes it easy to see what's been pulled from the image, without hunting around. You can upload several images at once, reprocess images if you need, and export the results to use later.

I kept the layout simple and responsive—no clutter, nothing overwhelming. Using tools like Streamlit or Tkinter, the interface stays interactive and updates results on the spot. In the end, the user interface connects people to all the complicated stuff

happening in the background and makes the whole system feel straightforward.

2.1.1.3 Hardware Interfaces

The hardware setup for this system is pretty straightforward. You don't need any fancy or expensive machines to get things up and running—just a regular laptop or desktop with average specs does the job. You'll use the usual keyboard and mouse to upload images and move around the application.

For output, any monitor or screen will work fine. If you want to speed things up, especially when you're training the model or dealing with lots of data, having more RAM and a decent GPU helps a lot. A GPU really cuts down the time it takes to train the YOLO model and makes everything run smoother.

Still, you don't need powerful hardware to use the system. The whole thing is built so people with basic equipment can use it, which is great for students or folks working on a small scale who don't have access to high-end gear.

2.1.1.4 Software Interfaces

The software interface pulls together all the key tools, libraries, and frameworks needed to build and run the system. For this project, Python takes center stage—it's easy to pick up, and it offers strong support for machine learning and image processing.

Different libraries handle different jobs. OpenCV handles image tasks like resizing, filtering, and cleaning up noise. When it comes to spotting objects, the YOLO model does the heavy lifting, offering fast and accurate detection that works well in real time. For picking text out of images, Tesseract OCR steps in to recognize and extract what's written.

To process the extracted data, the system brings in NumPy and Pandas. These libraries make it easy to manage and operate on data. The user interface ties everything together and comes to life with frameworks like Streamlit or Tkinter, connecting what's happening behind the scenes to what the user actually sees and interacts with.

All these components work in sync, making sure the system runs smoothly from

start to finish.

2.1.1.5 Functional Requirement

Here's what the system needs to do: take images from users and run them through a set process. As soon as someone uploads an image, the system jumps right into preprocessing to clean up the picture, making it easier to analyze.

Next, it scans the image for key objects using the trained YOLO model. When it spots something important, it pulls out any relevant text from those areas using OCR techniques. This text then goes through a check—looking for the right patterns, formats, and expected values, all based on predefined rules.

Once the system's done, it shows the results in a way that's easy to follow, including adding bounding boxes so you can actually see where things were found. Users can then export everything in formats like CSV or TXT if they want to work with the data elsewhere. If anything goes wrong—maybe an input isn't valid, or the image can't be read—the system doesn't just crash. Instead, it handles the error cleanly and explains to the user what happened.

2.1.1.6 User Characteristics

The system works for all kinds of users—students, researchers, or professionals dealing with image-based data. You don't need advanced tech skills to get started. The idea is to keep things simple and straightforward.

If you know how to upload files, click through menus, and read basic results, you're good to go. The interface is user-friendly, so even if it's your first time, you'll pick things up fast. There's no need for special training.

For people with a bit more technical experience, there's extra value in understanding what goes on behind the scenes. The system leaves room to dig a little deeper if you want to. So whether you're just starting out or you already have some technical know-how, you'll find the system flexible enough to suit your needs.

2.1.1.7 Constraints

There are certain constraints that may affect the performance and efficiency of the systm. One of the main limitations is related to the quality of the input images. If the images are blurred, badly lit, or contain distorted text, the accuracy of both object detection and OCR may decrease significantly.

Another thing to keep in mind: the models themselves have limits. The YOLO model only recognizes objects it's been trained on, so anything totally new just won't get picked up. For OCR, it really comes down to how clear and readable the text is. If the text in the image is a mess, the results won't be great.

Hardware matters too. If you're running this on a device that's not very powerful, processing images will slow down. Juggling several images at once makes it even tougher. All in all, you need decent input images and a reasonably good setup for the system to really shine.

2.1.1.8 Assumption and Dependencies

This system runs with a few assumptions in mind. For starters, users need to upload images that are clear enough so the models can spot objects and read text without much trouble. The images have to actually contain relevant objects and text the system can process.

A lot of the heavy lifting happens thanks to external tools like YOLO, which handles object detection, and Tesseract, which takes care of the OCR part. So, everything depends on these tools being installed and set up properly. That means Python and all the right libraries need to already be in place.

If you're missing libraries, used the wrong settings, or have version conflicts, things break down quickly. Keeping the setup straight and the environment tidy is the best way to make sure everything works the way it should.

Chapter 3
System Design Specification

3.1 System Architecture

The system architecture of this project brings all the different parts together in a way that just works—clean and organized. It uses a pipeline setup, so when you upload an image, it moves step-by-step through several stages. Each one focuses on a single job before handing things off to the next. This keeps things easy to follow, and honestly, it's great if you need to update or fix something later on.

It all starts with the user interface. You upload your image, and right away, that file heads to the backend. The first stop in the backend is preprocessing: the system cleans things up—resizes the image, normalizes it, and clears out any noise. The goal here is to make sure the image is in top shape before doing anything else.

Once the image is prepped, it goes to object detection. Here, the YOLO model steps in and hunts for key objects or regions. It draws bounding boxes around what matters, so the focus stays only on the important parts. These marked-up sections then get sent down the line.

Next, the OCR module gets to work. It uses Tesseract OCR to pull out any text from those detected areas. Sometimes this text isn't perfect, so the validation module follows up. This part applies specific rules to see if the text matches what's expected. Doing this makes sure the output is solid.

A main pipeline ties all these modules together, smoothly moving data from one part to the next. Once everything's done, the results head back to the user interface—now with bounding boxes and the extracted, validated text displayed. Users get a clear, easy-to-read result.

In the end, this setup keeps everything running smoothly and efficiently, making it easy to process and analyze images automatically.

3.2 Module Decomposition Description

The system breaks down into separate modules, each handling its own job. This setup keeps things organized. It's easier to build and maintain because every module zeroes in on one function, but they all stay linked by a main pipeline that connects everything.

It starts with preprocessing. This module takes the raw input image and gets it ready—resizing, normalizing, clearing out basic noise. It's pretty important, honestly, since cleaner input means you'll get much better results later.

Next comes object detection. Here, a YOLO-based model hunts for key objects or regions, boxing them out so the system focuses only on what matters.

Once that's done, the OCR module jumps in. It uses Tesseract OCR to pull text from those selected spots. Let's be real—sometimes the text has errors thanks to the image quality or weird fonts. That's when the validation module steps in. It checks the text formatting and makes sure it meets whatever rules are required. That keeps things accurate and reliable.

All these pieces talk to each other through a main pipeline controller, making sure data flows smoothly from one module to the next. At the end of it all, there's a user interface. Here, people can upload images, see results, and export outputs if they want.

Splitting the system into modules just makes everything clearer and runs smoother. Plus, it's way simpler to upgrade or add new features down the line.

3.3 High Level Design Diagrams

The high-level design lays out how everything in the system connects and works together. You get a real sense of how input moves through each step, without getting bogged down in the small stuff.

Here's how it flows in this project: The user uploads an image through the app. That image first goes to the preprocessing module, which gets it ready for the next stage. Then, it moves on to object detection, where the system uses YOLO to find

the important regions.

Once those regions are picked out, the OCR module jumps in and pulls out any text it finds. The system then checks that text to make sure it fits the rules. When that's done, the final results pop up on the user's screen.

The diagram just strings these pieces together in order, giving a clear, straightforward look at how the whole system works.

3.3.1 Use Case Diagram

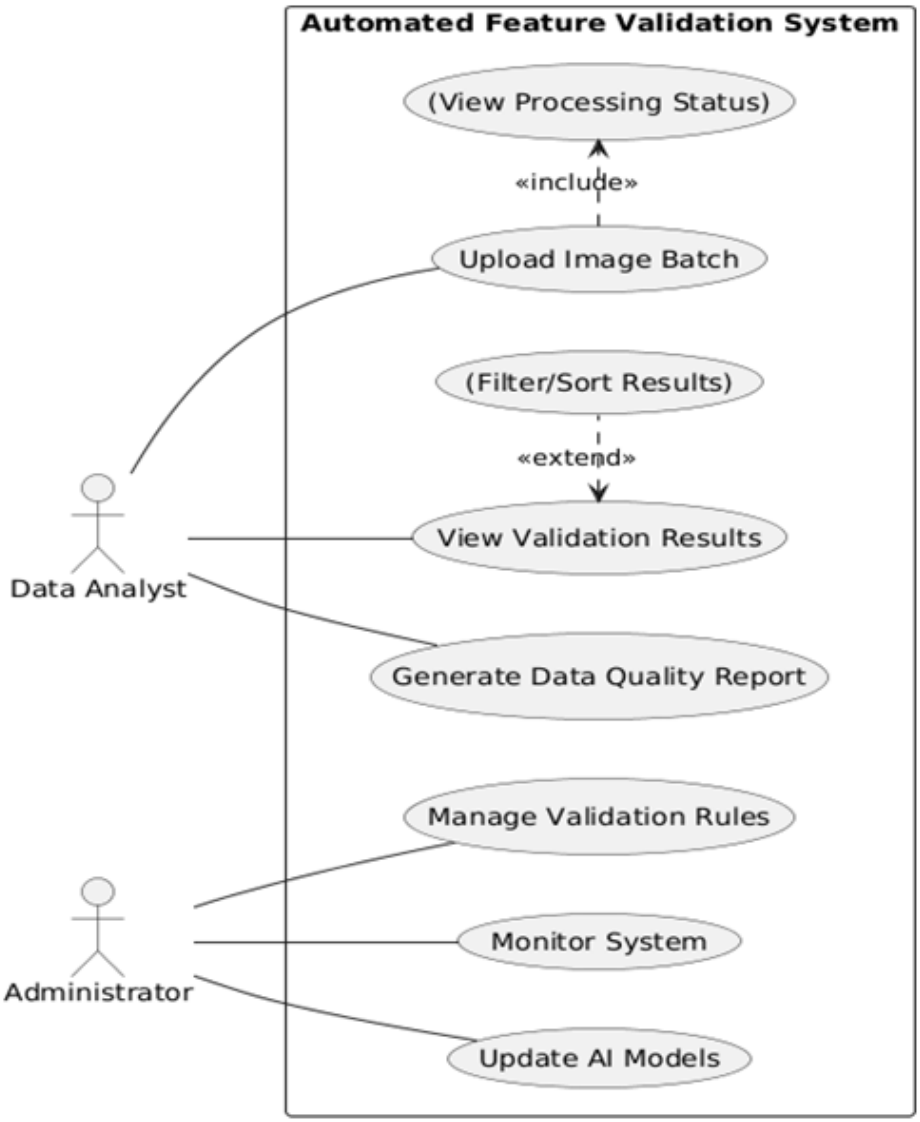


Figure 3.1: Use Case diagram

The Use Case Diagram shows how people interact with the "Automated Feature Validation System." There are really just two main roles in this setup: Data Analyst

and Administrator.

The Data Analyst uploads batches of images, checks the processing status, and reviews validation results. After uploading, the system steps in—it preprocesses the images, runs object detection, does OCR extraction, and handles validation. Once that's done, the Analyst sees the results. If needed, they can filter or sort the results and even generate data quality reports from all the information the system pulls out.

Meanwhile, the Administrator manages the system's backend. They set up and update validation rules keep an eye on how the system is running, and handle AI model updates, like switching out the YOLO detector or adjusting OCR settings.

So, really, the diagram just lays out who does what—who uploads and analyzes data, and who keeps things running under the hood. It's all about making sure the right people have control, both on the user side and behind the scenes.

3.3.2 Data-Flow Diagram

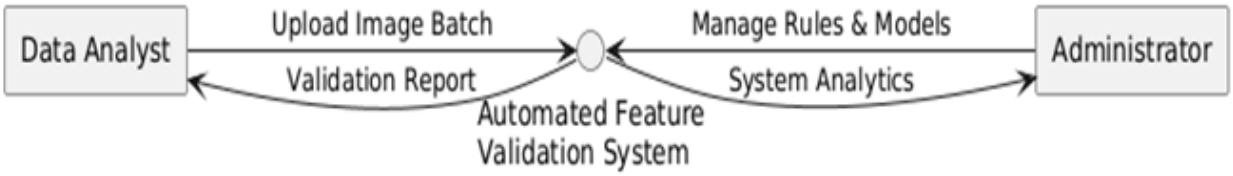


Figure 3.2: DFD-0

The Data Flow Diagram (DFD) shows how data moves through the system. At the top level (DFD-0), the Data Analyst sends in batches of images, and the Automated Feature Validation System handles the processing Once that's done, the system sends back validation reports. The Administrator steps in to manage rules and models and gets analytics from the system as the final output.

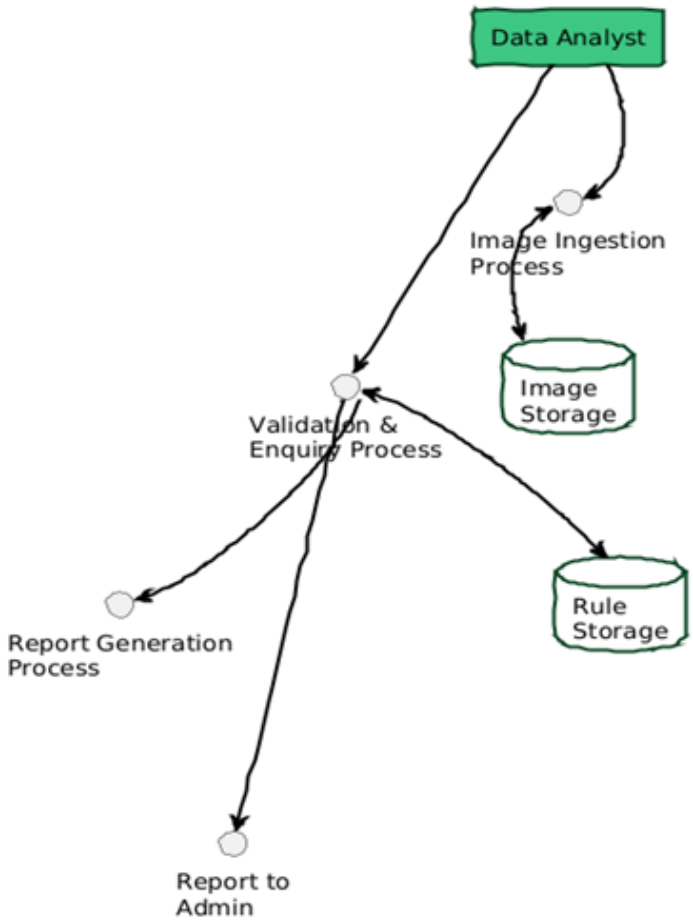


Figure 3.3: DFD-1

At the detailed level (DFD-1), things kick off with image ingestion basically, uploading images and saving them in storage. From there, the images head to validation and extraction. That's where the real work happens: preprocessing, detecting objects, and running OCR to pull out the data. Then, the system checks what it found against the rules saved in rule storage.

Once validation's done, the process moves to report generation. Here, it pulls everything together into a structured report In the end, the system delivers that report straight to the administrator or user. The whole flow lays out exactly how data moves through each stage, from storage to processing, showing every step along the way.

3.3.3 Class Diagram

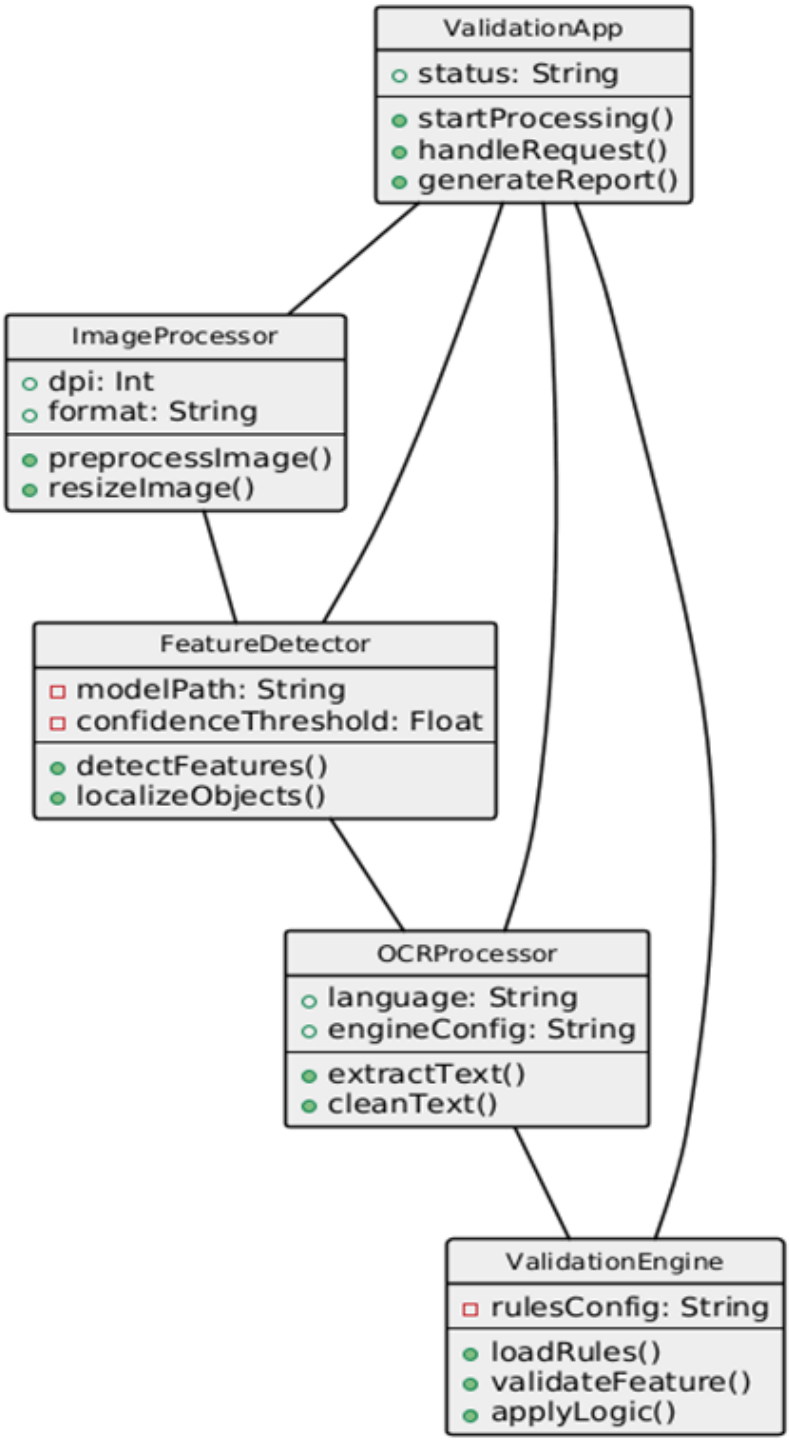


Figure 3.4: Class Diagram

The Class Diagram lays out how the system is organized. At the center, you've got the Validation App class calling the shots—managing the whole workflow and handling image processing from start to finish. It works with other classes like Im-

ageProcessor, FeatureDetector, OCRProcessor, and ValidationEngine.

Image Processor takes care of the basics—resizing images and cleaning them up. Feature Detector uses the YOLO model to spot objects and pick out key features in each image. OCR Processor jumps in next, pulling out text from the regions that were found and then cleaning up that text.

Then comes Validation Engine. This class checks the extracted data against a set of rules, making sure everything lines up as it should. Each class has a clear job, and when you put it all together you get a streamlined pipeline for processing images. The class diagram spells out how everything connects—so you can see how data moves through the system and how each part fits into the bigger picture.

3.3.4 Sequence Diagram

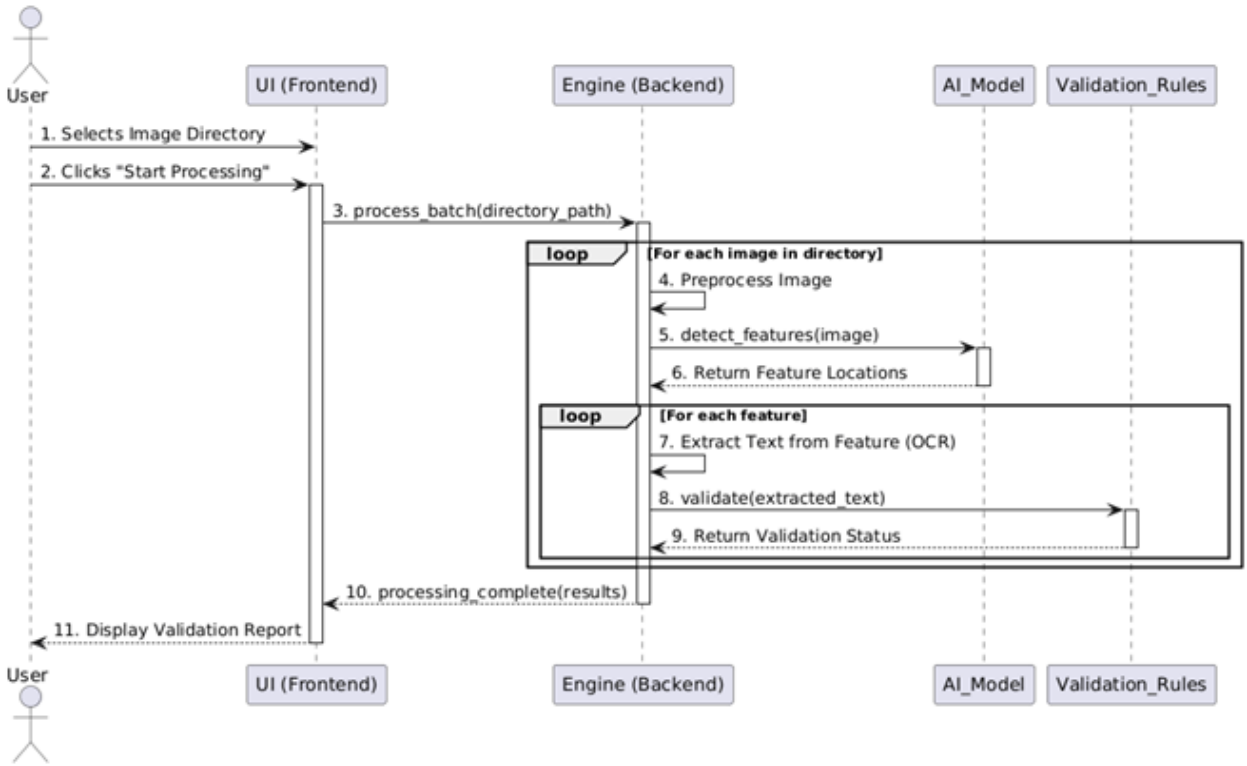


Figure 3.5: Sequence Diagram

The sequence diagram lays out how everything talks to each other as the system runs. Here's how it all kicks off: the user picks a folder of images and hits start in the interface. That request heads straight to the backend engine.

The backend rolls up its sleeves and works through each image, one by one.

First, it cleans up the image to boost quality. Then the AI model spots key features and pinpoints where they are. Wherever it finds something it runs OCR to pull out any text.

That text isn't just taken at face value it goes through the validation rules module, checking if everything looks right. The module sends back a pass or fail, and the backend keeps moving through the images When it's done, the system puts all the results together and creates a validation report. The user sees this report in the interface.

So, this diagram doesn't just show what happens it actually maps out who talks to whom at each step.

Chapter 4
Methodology and Team

4.1 Introduction to Agile Methodology

Agile is really about staying adaptable and making sure the whole team moves together. Instead of trying to plan every last detail up front and sticking with it no matter what, you break the work down into smaller chunks. You tackle each piece in these short rounds yeah, those sprints everyone talks about. During each sprint, the team builds something, tests it, checks if it's working, and then figures out what to do next. If something goes wrong, you catch it early before it turns into a giant mess. There's always progress step by step so you can literally see the project taking shape as you go.

Here's the real kicker: feedback is built into every step. The team talks things through, and stakeholders even actual users jump in and share what's working and what isn't. All that back-and-forth really helps the final product line up with what people actually want, and, honestly, it saves everyone from those last-minute disasters. You've got regular check-ins, so nobody ends up in the dark or left dealing with problems that could've been fixed earlier.

That's how we handled this project. We took it one module at a time preprocessing, object detection, plugging in OCR, the whole validation routine. We'd build a piece, give it a try, fix what didn't work, and then move on. It paid off big time: we caught mistakes early, got the models running better, and made sure everything played nice together.

In the end, Agile is just the way to go when you're dealing with complicated projects that shift around a lot, like field image analysis. It keeps the process moving smoothly, and you end up with something you can actually trust.

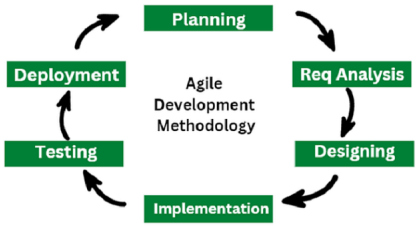


Figure 4.1: Agile model

The sequential phases in Agile model are-

1. **Requirement Gathering:** First, we figured out exactly what this project needed. The main goal was to automate image analysis and validation. All of these matched what the project asked for and what we expected the system to do..

2. **Design:** Next, we planned how everything would fit together. We broke the workflow into different modules preprocessing, object detection with YOLO, OCR extraction, and validation. We also mapped out the user interface and how data would move between pieces of the system, making sure everything would play nicely together.

3. **Development (Coding):** Then came building the thing. We used Python for all modules: processing the images, running YOLO for object detection, integrating Tesseract for OCR, and writing the validation logic. We put together the main pipeline script, built the user interface, and got all the parts talking to each other.

4. **Testing:** We tested the system on a variety of images to check how well detection and text extraction worked. Every time we ran into issues—like bad image quality, wrong OCR output, or problems with validation—we fixed them. Running different test cases helped us iron out bugs and make the system more reliable.

5. **Deployment:** After things were running smoothly, we deployed the system locally. The user interface let people upload images and see their results right away. We ran the whole pipeline as one piece to make sure every module worked together with no surprises.

6. **Maintenance:** We kept an eye on the system after deployment. We tuned the models, improved preprocessing, and fixed any hiccups that popped up. Updates weren't just about bug fixes—each change made the system easier to use and more efficient overall.

4.2 Team Members, Roles & Responsibilities

MEMBER 1: Prachi Shrimali		NAME OF MODULE: Data Curation & Preprocessing	
NAME OF FUNCTIONALITY	SOFT DEADLINE DATE	HARD DEADLINE DATE	DETAILS OF FUNCTIONALITY
Develop Preprocessing Scripts	08-Aug-2025	18-Aug-2025	Write initial OpenCV scripts for image resizing & grayscale conversion.
Implement Image Enhancement	26-Sep-2025	06-Oct-2025	Add functions for noise reduction and contrast enhancement.
Create Annotation Scripts	21-Oct-2025	31-Oct-2025	Write helper scripts to read/write annotation data (bounding boxes).
Develop Data Augmentation	05-Feb-2026	15-Feb-2026	Create scripts to artificially expand the dataset (rotate, flip, scale).
Finalize Preprocessing Module	10-Mar-2026	20-Mar-2026	Integrate all scripts into a single, optimized preprocessing class.
Module Unit Testing	05-Apr-2026	15-Apr-2026	Write & execute tests to verify all preprocessing functions work correctly

Table 4.1: Member-1 Roles and Responsibilities

MEMBER 2: Manuraj Singh		NAME OF MODULE: System Integration & Backend	
NAME OF FUNCTIONALITY	SOFT DEADLINE DATE	HARD DEADLINE DATE	DETAILS OF FUNCTIONALITY
Design System Architecture	08-Aug-2025	18-Aug-2025	Create a flowchart for how all modules will connect and interact.
Develop Main Pipeline Script	26-Sep-2025	06-Oct-2025	Write the main app.py script that controls the overall data flow.
Integrate Preprocessing Module	21-Oct-2025	31-Oct-2025	Create an API/function to call the preprocessing module from the pipeline.
Integrate AI Models	05-Feb-2026	15-Feb-2026	Create APIs to load and get predictions from the trained models.
Implement Backend Validation	10-Mar-2026	20-Mar-2026	Code the final rules (regex, format checks) to validate the OCR output.
Integration & API Testing	05-Apr-2026	15-Apr-2026	Write tests to ensure all modules communicate correctly within the pipeline.

Table 4.2: Member-2 Roles and Responsibilities

MEMBER 3: Piyush Kewlani		NAME OF MODULE: AI Model Development & Training	
NAME OF FUNCTIONALITY	SOFT DEADLINE DATE	HARD DEADLINE DATE	DETAILS OF FUNCTIONALITY
Setup Training Environment	08-Aug-2025	18-Aug-2025	Configure GPU environment; install TensorFlow/PyTorch & dependencies.
Implement Data Loaders	26-Sep-2025	06-Oct-2025	Write scripts to load annotated data and feed it to the model.
Train Baseline Detection Model	21-Oct-2025	31-Oct-2025	Execute the first training run for the YOLO model with default settings.
Implement OCR Fine-Tuning Script	05-Feb-2026	15-Feb-2026	Write script to re-train Tesseract on a custom character set if needed.
Hyperparameter Tuning	10-Mar-2026	20-Mar-2026	Optimize model parameters (learning rate, etc.) for best accuracy.
Model Accuracy Testing	05-Apr-2026	15-Apr-2026	Evaluate final models against the test dataset and log performance metrics.

Table 4.3: Member-3 Roles and Responsibilities

MEMBER 4: Mitesh Jangid		NAME OF MODULE: UI Development & Testing	
NAME OF FUNCTIONALITY	SOFT DEADLINE DATE	HARD DEADLINE DATE	DETAILS OF FUNCTIONALITY
UI/UX Mockup Design	08-Aug-2025	18-Aug-2025	Create visual wireframes and mockups for the application layout.
Develop Basic UI Frontend	26-Sep-2025	06-Oct-2025	Build the initial UI with Streamlit/Tkinter (e.g., file upload button).
Connect UI to Backend	21-Oct-2025	31-Oct-2025	Link the UI buttons to the backend pipeline functions.
Implement Result Display	05-Feb-2026	15-Feb-2026	Write code to display the final image with bounding box and text in the UI.
Implement Data Handling/Export	10-Mar-2026	20-Mar-2026	Add functionality to the UI to save results to a file (CSV/TXT).
User Acceptance Testing (UAT)	05-Apr-2026	15-Apr-2026	Perform end-to-end testing from a user's perspective to find bugs.

Table 4.4: Member-4 Roles and Responsibilities

Chapter 5
System Testing

5.1 Introduction

System testing really drives this project forward. At this point, it's all about making sure every moving part—preprocesing, object detection, OCR, validation, the user interface—actually works together as a single, solid system.

We don't just test with perfct images. Some are sharp, but plenty look rough—bad lighting, blury edges, weird angles, you name it. The point isn't to break the system, but to see if it can handle real-world messiness. Can it still spot the right objects? Does it grab the right text? Are the results still good all the way through?

Watching each module alone isn't enough. We pay close attention to how data moves from one stage to the next. Does every piece get passed along just as it should? Or does somthing glitch out somewhere in the chain? If the process stumbles, we know where to dig in and fix things.

And if the user interface doesn't work? All that backend effort doesn't mean much. So, we make sure users can upload images easily, get instant feedback, and export everything without running into annoying problems.

So, system testing isn't just about squashig bugs. It's about making sure every part holds up the whole system runs smoothly, and people actually enjoy using it when it's finally out there.

5.2 Types of Testing Performed

This project puts a big focus on testing to make sure everything works way it should, every step of the way. The system isn't just one big piece—it's built from several modules like preprocessing, object detction, OCR, validation, and the user interface. So just testing the whole thing once isn't enough; you have to look at each part on

its own, too.

We test object detection to see if it actually finds what it's supposed to. We also check if the system extracts text correctly and if the validation rules catch mistakes. That's not all—we also keep an eye on how these modules talk to each other. Smoth data flow matters, because even if each part works, the whole thing falls apart if they don't connect properly.

By running these tests early and often, we spot problems before they grow and make sure the system keeps running smoothly.

5.2.1 Unit Testing

Unit testing is implemented to verify whether individual modules function properly and perform their tasks. Each system module used in the project will be separately verified for performing its own role prior to integrating with other components. This will help to detect any problems early and improve the reliability of the developed system.

1. **Image Input and Preprocessing Module**:unit testing should check how effectively it loads images from different files and sources and preprocesses them by resizing and normalizing or removing noise. Additionally, data augmentation operations should be performed correctly, with valid variations generated but not distorting essential features of the picture.

2. **Object Detection Module**:The focus here is going to be on verifying the quality of the YOLO model and ensuring that it can detect the objects on pictures successfully. To do so, this module will be checked using several sample images and examining whether detected objects have been highlighted with proper bounding boxes.

3. **OCR and Text Extraction Module**:Unit testing of the OCR and Text Extraction Module will include checking how text is extracted from detected objects. Different levels of image clarity will be used to test the efficiency of the operation, which will include cases when the text is difficult to read.

4. **Validation Module**:Validation Module is designed to verify text extracted by OCR and to exclude any invalid values according to defined rules. Therefore, unit testing of this module should involve testing whether the module checks text according to certain rules, for instance, length or specific patterns.

5. **User Interface Module**:unit testing of the User Interface Module should cover all features included in the interface. It will include testing whether the image upload works correctly, images with text and object detections can be shown with proper bounding boxes, and finally, whether the output can be exported.

By means of thorough unit testing of the modules, each component can be checked for correct functioning separately.

5.2.2 Integration Testing

The aim of conducting integration testing for the current project was to ensure that the different parts of the system work properly together to form an effective pipeline. Considering the various stages involved in the process of operation such as pre-processing, object detection, OCR validation, and the user interface stage, it becomes necessary to test the process for any loss of information in transitioning from one stage to another.

Testing was done to ensure that the output of a particular stage was provided to the next stage as its input.

1. **Image Preprocessing with Object Detection:**For evaluating this aspect, pre-processed images were sent to the object detection module. The aim was to confirm that the procedures followed during image processing did not affect the ability to detect objects. As expected, the system confirmed that the YOLO model detects objects within the preprocessed images effectively.

2. **Object Detection and OCR Integration:** This particular integration was tested to establish whether the detected regions were properly extracted for text recognition. The system ensures that only relevant sections of the image are sent to

the OCR module, after which the latter extracts texts from the images. Successful extraction shows that integration was done effectivey.

3. **OCR Module and Validation Module Integration:** In this case, text extracted by the OCR module is provided to the validation module to evaluate if there is adhernce to the established requirements and guidelines. The system was tested for the proper filtering out of irelevant information while accepting valid information from the OCR module.

4. **End-to-End Integration:** For assessing the functionality of the complete pipeline flow, the whole system was considered. In this regard, a user uploads an image, which undergoes processes such as pre-processing, detection, text extraction, and validation. Results show that this process was conducted effectively in the system.

5. **Backend and User Interface Integration:** This integration aims at ensuring that outputs of the backend modules are shown in the frontend correctly. The features under assessment include uploading an image, displaying bounding boxes, extracted texts, and exporting them. Results show that these actions are executed correctly and in time.

6. **Error Handling Integration:** For error handling tests, some invalid and poor quality images were uploaded. The purpose of this test was to examine the performance of the system when a certain module returns erroneous output. This evaluation confirms that the system does not collapse but provides appropriate outputs.

5.2.3 System Testing

System testing will help to test the system as a whole and to ensure that it provides the desired functionalities. In this phase, all the modules will be tested as a whole to see if they work together efficiently and effectively from the input stage to the output stage.

From the perspective of this project, system testing will require different inputs

in terms of image types to monitor the functioning of the system as a whole from preprocessing to object detection and then to OCR and finally to validation. The aim of this testing will be to determine if the system can provide accurate and meaningful outputs for any image type fed into the system.

Furthermore, this testing will help to ensure that the system performs adequately regardless of the image qualities, environment, or light. In addition, the user interface will also be tested to see if the user can upload and view images as well as export the data from the system successfully.

To conclude, system testing will provide confirmation regarding system functionality and stability as well.

5.3 Test Cases

NAME OF MODULE:Input & Preprocessing Module			TESTING TOOL :Python, Manual Testing	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC01	Check image upload	Valid image (JPG/PNG)	Image should load successfully	Image loaded successfully
TC02	Validate image resizing	Image of different sizes	Image resized to standard size	Resized correctly
TC03	Test normalization	Raw image	Pixel values normalized	Normalization done
TC04	Test noise removal	Noisy image	Noise reduced, clarity improved	Image enhanced
TC05	Test data augmentation	Input image	Augmented images generated	Generated successfully
TC06	Annotation script test	Annotated dataset	Annotations created correctly	Generated correctly
TC07	Invalid image handling	Corrupted file	Error message shown	Handled properly

Table 5.1: Test Case table for Image Input and Preprocessing Module

NAME OF MODULE:Object Detection Module			TESTING TOOL :YOLO, Python	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC07	Check object detection	Preprocessed image	Objects detected with bounding boxes	Detected correctly
TC08	Check detection accuracy	Image with known objects	Correct objects identified	Accurate detection
TC09	Test multiple objects	Image with multiple objects	All objects detected	Multiple objects detected
TC10	No object case	Image without objects	No detection	No objects detected
TC11	Low quality image	Blurry image	Partial detection	Reduced accuracy

Table 5.2: Test Case table for Object Detection Module

NAME OF MODULE:OCR & Text Extraction Module			TESTING TOOL :Tesseract OCR	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC12	Extract text	Image with clear text	Correct text extracted	Extracted correctly
TC13	Unclear text test	Blurred text image	Partial text extraction	Extracted partially
TC14	Multiple text regions	Image with multiple text	All text extracted	Extracted successfully
TC15	No text case	Image without text	No text output	No output
TC16	Special characters	Image with symbols	Correct extraction	Handled correctly

Table 5.3: Test Case table for OCR Module

NAME OF MODULE:Validation Module			TESTING TOOL :Python Rules	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC17	Validate correct text	Valid text	Accepted as valid	Accepted
TC18	Invalid format test	Wrong pattern text	Rejected	Rejected
TC19	Empty input test	No text input	Error message	Handled correctly
TC20	Pattern matching	Formatted text	Matches pattern	Validated
TC21	Mixed data test	Valid + invalid text	Only valid accepted	Filtered correctly

Table 5.4: Test Case table for Validation Module

NAME OF MODULE:System Integration Module			TESTING TOOL :End-to-End Pipeline	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC22	Full pipeline test	Valid image	Complete output generated	Working correctly
TC23	Data flow check	Input image	Smooth module flow	No errors
TC24	Multi-step validation	Complex image	Correct detection+OCR+validation	Successful
TC25	Error handling test	Invalid input	Handled without crash	Stable output

Table 5.5: Test Case table for Integration Module

NAME OF MODULE:User Interface Module			TESTING TOOL :Streamlit/Tkinter	
Test ID	**Test Description**	**Input Data**	**Expected Output**	**Actual Output**
TC26	Upload image	Valid image	Image uploaded successfully	Uploaded
TC27	Display results	Processed image	Bounding boxes + text shown	Displayed correctly
TC28	Export results	Output data	File downloaded	Export successful
TC29	Invalid upload	Wrong file type	Error message	Handled correctly
TC30	UI responsiveness	User interaction	Smooth working	Responsive

Table 5.6: Test Case table for UI Module

Chapter 6

Project Screen Shots

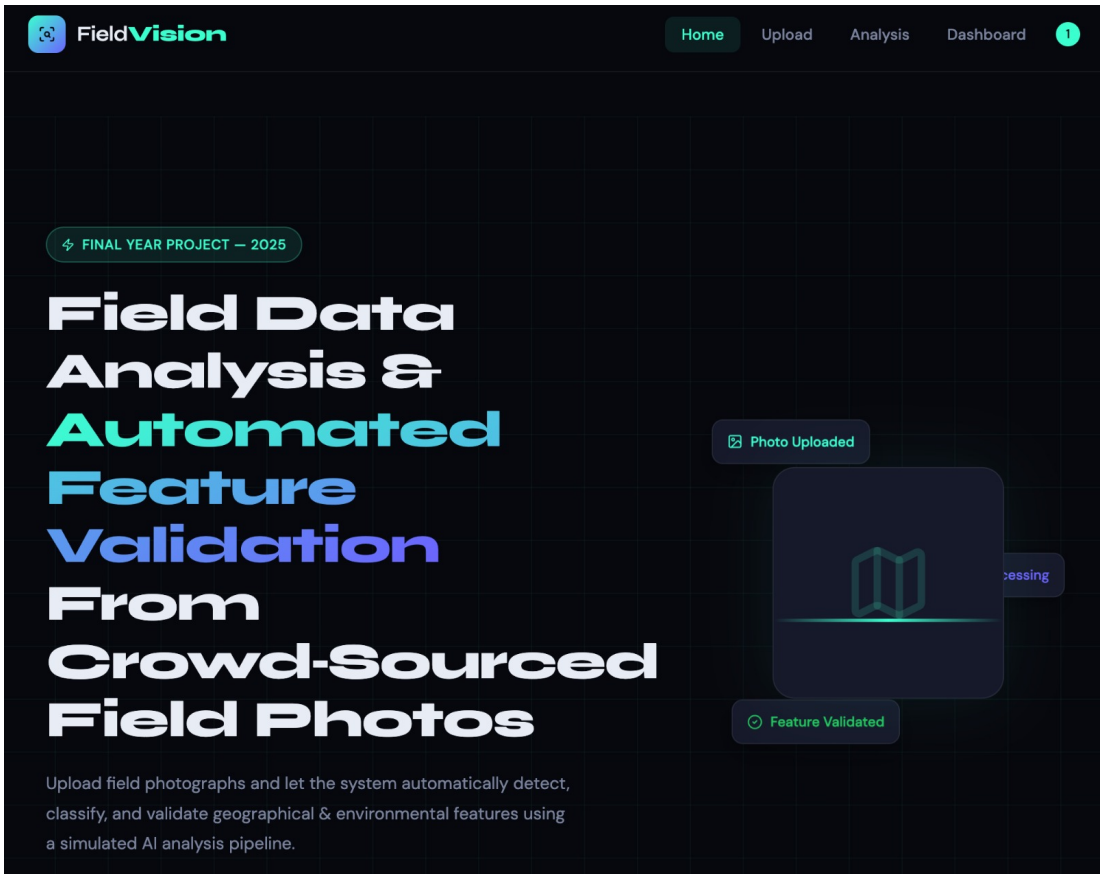


Figure 6.1: Home Page

The home page is your first stop in the FieldVision system. Right away, it shows what the project's all about: automatically analyzing and validating features from photos people share from the field. The layout keeps things simple and clear, with a straightforward three-step guide—capture or upload your images, let the system handle the analysis, then check the results and export them if you want. You'll see easy-to-find navigation buttons like Home, Upload, Analysis, and Dashboard, so it's simple to jump between sections. Before you dive in, the home page lays out the workflow so it all makes sense, helping everything run smoothly from the start.

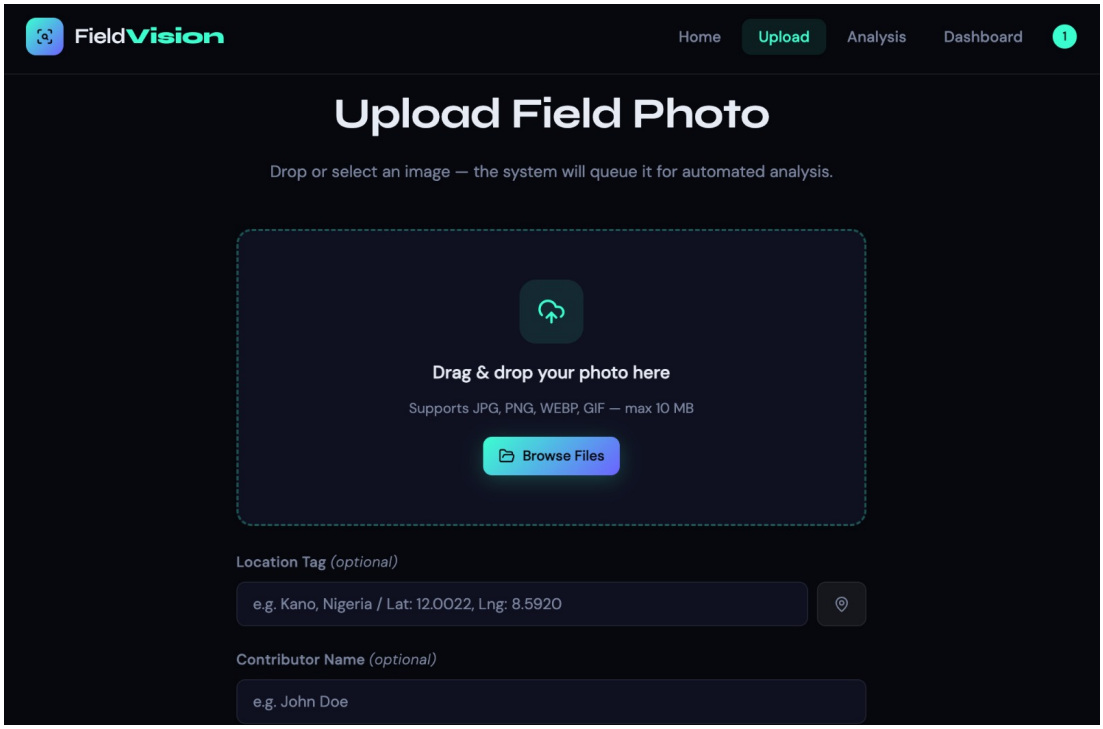


Figure 6.2: Image Upload Interface

The upload interface makes it easy for users to add images for processing. You can just drag and drop your files or browse your computer to find them. It works with popular formats like JPG, PNG, and WEBP, so there's no need to worry about compatibility. If you want, you can add extra details like where the photo was taken or your name as the contributor—that way, the uploads have more context. Once you upload an image, the system takes over and runs it through steps like preprocessing, object detection, and pulling out any text using OCR. The whole setup aims to be straightforward, even if you're not tech-savvy. This upload module really kicks off everything that happens next in the system so it's a key piece of the workflow.

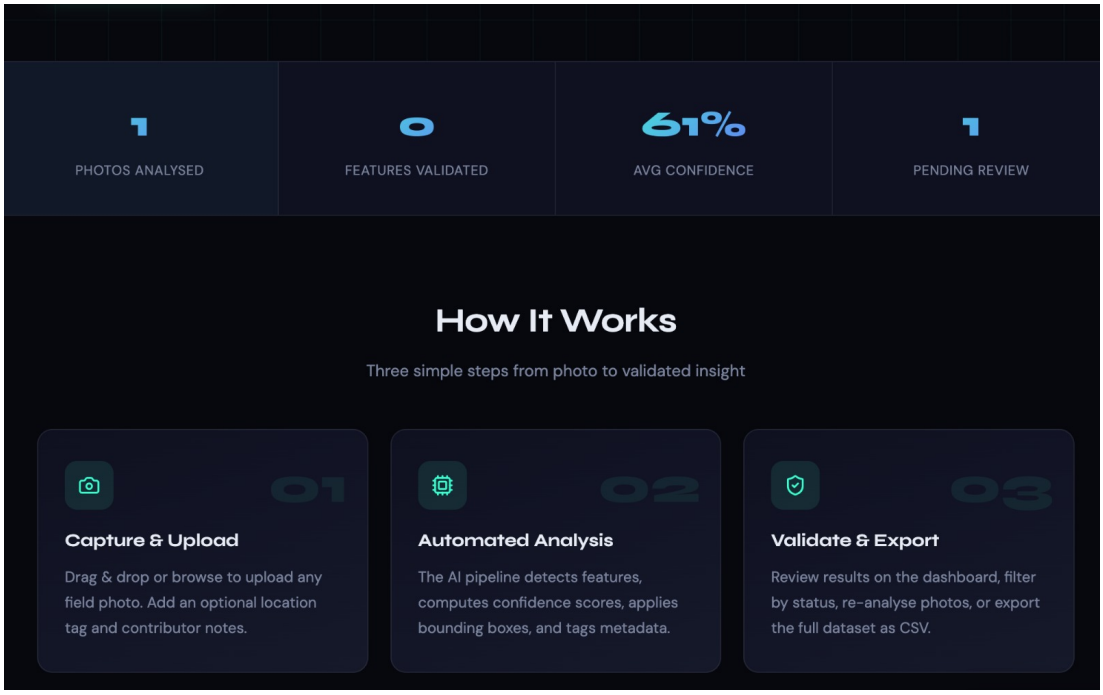
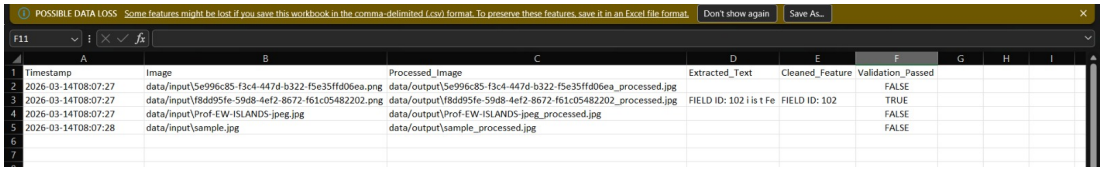


Figure 6.3: System Dashboard and Analytics View

The dashboard provides a summarized view of the system's performance and processing results. It displays important metrics such as the number of photos analyzed, features validated, average confidence score, and pending reviews. These metrics help users quickly understand how well the system is performing and whether further action is needed. The dashboard is designed to give a quick overview without going into detailed results, making it useful for monitoring purposes It also reflects real-time updates based on the images processed by the system. By presenting the data in a structured and visual format, the dashboard helps users make quick decisions and track the overall progress of the analysis process.



POSSIBLE DATA LOSS Some features might be lost if you save this workbook in the comma-delimited (.csv) format. To preserve these features, save it in an Excel file format. Don't show again Save As...

F11

	A	B	C	D	E	F	G	H	I
1	Timestamp	Image	Processed_Image	Extracted_Text	Cleaned_Feature	Validation_Passed			
2	2026-03-14T08:07:27	data/input\5e996c85-f3c4-447d-b322-f5e35ffd06ea.png	data/output\5e996c85-f3c4-447d-b322-f5e35ffd06ea_processed.jpg			FALSE			
3	2026-03-14T08:07:27	data/input\f8dd95fe-59d8-4ef2-8672-f61c05482202.png	data/output\f8dd95fe-59d8-4ef2-8672-f61c05482202_processed.jpg	FIELD ID: 102 i is t Fe	FIELD ID: 102	TRUE			
4	2026-03-14T08:07:27	data/input\Prof-EW-ISLANDS-jpeg.jpg	data/output\Prof-EW-ISLANDS-jpeg_processed.jpg			FALSE			
5	2026-03-14T08:07:28	data/input\sample.jpg	data/output\sample_processed.jpg			FALSE			
6									
7									

Figure 6.4: Result Output and Export Data View

Here's where you see the system's final results after it finishes processing your images. Everything pops up in a table laying out details like the timestamp, input and processed image paths, the text it pulled from the image, cleaned features, and a validation column. That validation status? It tells you right away if the system followed the rules or messed something up, so you can spot errors without digging. If you need to keep working with this data, you can export the table as a CSV for repors or deeper analysis later. This part of the tool matters—it ties the whole workflow together and makes sure the output is clear, reliable, and ready to use.

Chapter 7
Conclusion and Future Scope

7.1 Conclusion

In summary, this project seeks to build a system that automatically analyzes field images. As the name implies, the system will detect objects within images and then perform Optical Character Recognition (OCR). Its goal is to turn unstructured images into something usable and organized through the process of image procesing, object detection, and text extraction Modules for all these operations were successfully built during this project.

The image analysis process involves the sequential application of functions such as image enhancement, region identification, text extraction, and validation of results. In this system, YOLO and Tesseract OCR have been used successfully to achieve this goal effectively. YOLO enables the proper detction of relevant features while Tesseract OCR ensures that all relevant text is extracted from images accurately.

Various tests have been performed on the system to ensure its effectiveness. From the result, it can be concluded that the system works very well, producing accurate results as expected. However, the quality of the input plays a major role in the output of the system.

From this project, it is evident that there are many areas where AI can be utilized. The current system has been built successfully; therefore, more improvements can be made in the future.

7.2 Future Scope

Further improvement can be attained by incorporating other advanced deep learning frameworks for object detection and text recognition tasks. Accuracy will be

improved, especially when the images have been blurred, noisier, or dark.

- In addition to the offline version of the system, the system may be developed in a real-time application for use either through a website or through a mobile device, allowing users to acquire images using the device they carry, thus eliminating the need for an auxiliary machine.
- Other custom-trained OCR models can be developed specifically for certain text types to improve text extraction accuracy.
- Complex validations may be added to the model to enable it to handle more complex formats, reducing the likelihood of errors when extracting information from images.
- Cloud computing can be used to host the software and database for the purpose of better handling of large volumes of data and record keeping and tracking changes made to the records.
- Training can be done on more data to help improve the accuracy and efficiency of the algorithm.
- Other functionalities can be added to enhance the user experience, such as batch processing of images, results history, and better visualization in the user interface.

Chapter 8
UN Sustainable Development Goals

8.1 Mapping with UN sustainable development goals

Goal 9: Industry, Innovation, and Infrastructure. The project uses state-of-the-art technology that includes artificial intelligence, object detection, and optical character recognition to create an automatic field data analysis solution. The use of innovations in the form of machine learning and other computer vision techniques helps convert unstructured imagery data into structured information.

Goal 11: Sustainable Cities and Communities. Applications include practical activities like field surveys, infrastructure monitoring, and data validation, thus helping in making cities smarter and creating effective solutions. Enhanced efficiency and accuracy of fieldwork contribute to proper planning and management of territories, whether urban or rural.

Goal 8: Decent Work and Economic Growth. By automating repetitive and boring processes associated with manual work, like data entry and data verification, this initiative helps increase productivity. At the same time, it stimulates organizations to integrate new technology into their operations and help people make better decisions.

Goal 12: Responsible Consumption and Production. The project helps decrease dependence on paper for collecting and validating data. The usage of images for digitization of the data makes it easier for individuals and organizations to operate with minimal negative impact on the environment.

It is clear that these mapping connections show how our project contributes to sustainable development of the world by stimulating innovation and efficiency as well as supporting the creation of smart infrastructures.

SDG Title: Industry, Innovation and Infrastructure		
S.No	**Objective**	**Outcome**
1	Automate analysis of field images using AI techniques	Reduces manual effort and improves efficiency in data processing
2	Improve accuracy of data extraction from real-world images	Provides reliable and structured information from unstructured data
3	Use advanced technologies like object detection and OCR	Promotes innovation and adoption of modern AI-based solutions
4	Develop a user-friendly system for easy accessibility	Enables users to interact with technology without technical complexity

Table 8.1: SDG Mapping

References

[1] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill Education, 2014.

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[4] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.

[5] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Morgan Kaufmann, 2011.

[6] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *International Conference on Learning Representations (ICLR)*, 2015.

[7] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems (NIPS)*, 2012.

[8] F. Chollet, *Deep Learning with Python*. Manning Publications, 2017.

[9] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[10] Google, "MediaPipe: Cross-platform ML Solutions," Available: https://mediapipe.dev/

[11] United Nations, "Sustainable Development Goals," Available: https://sdgs.un.org/goals

Project Poster

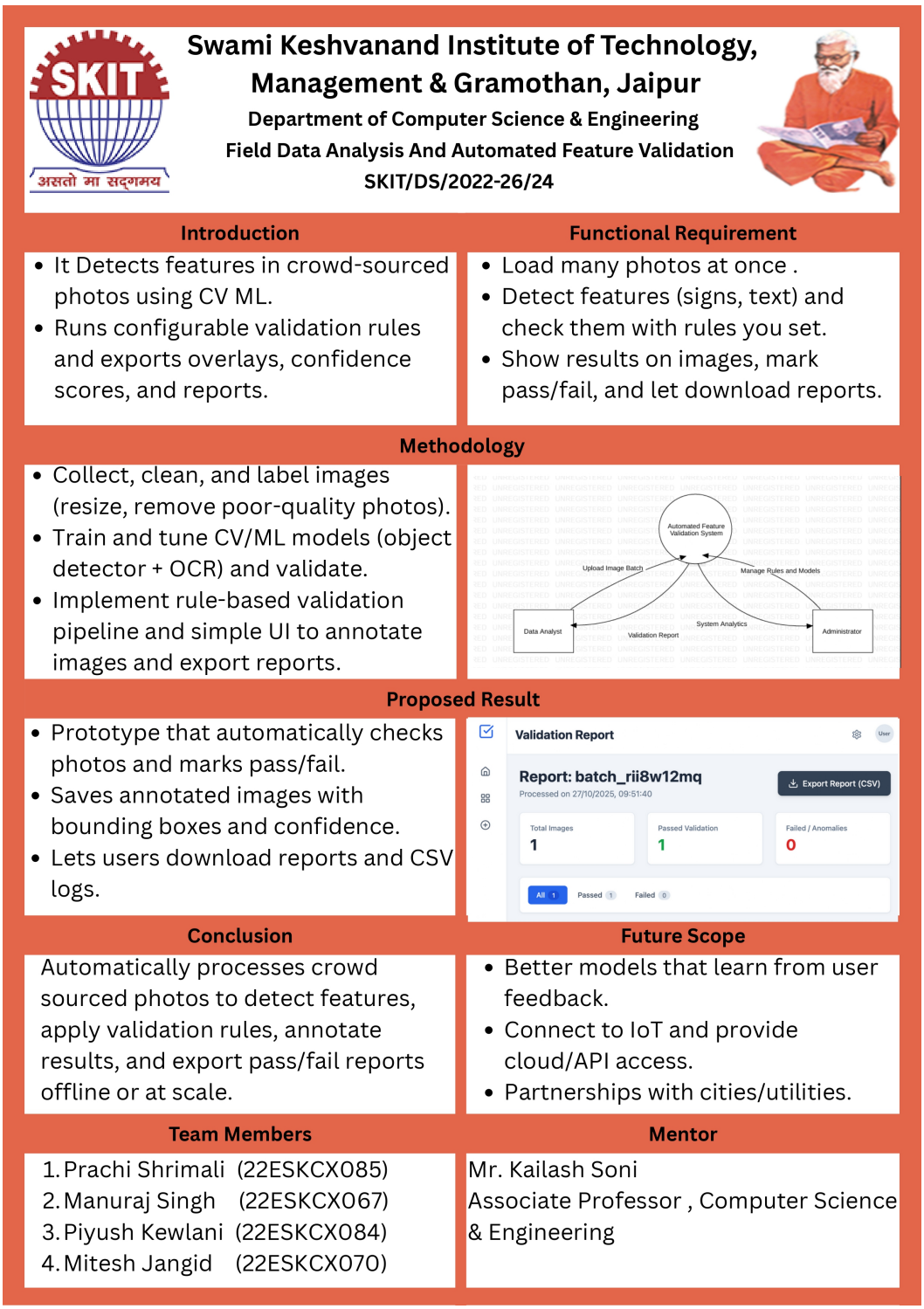
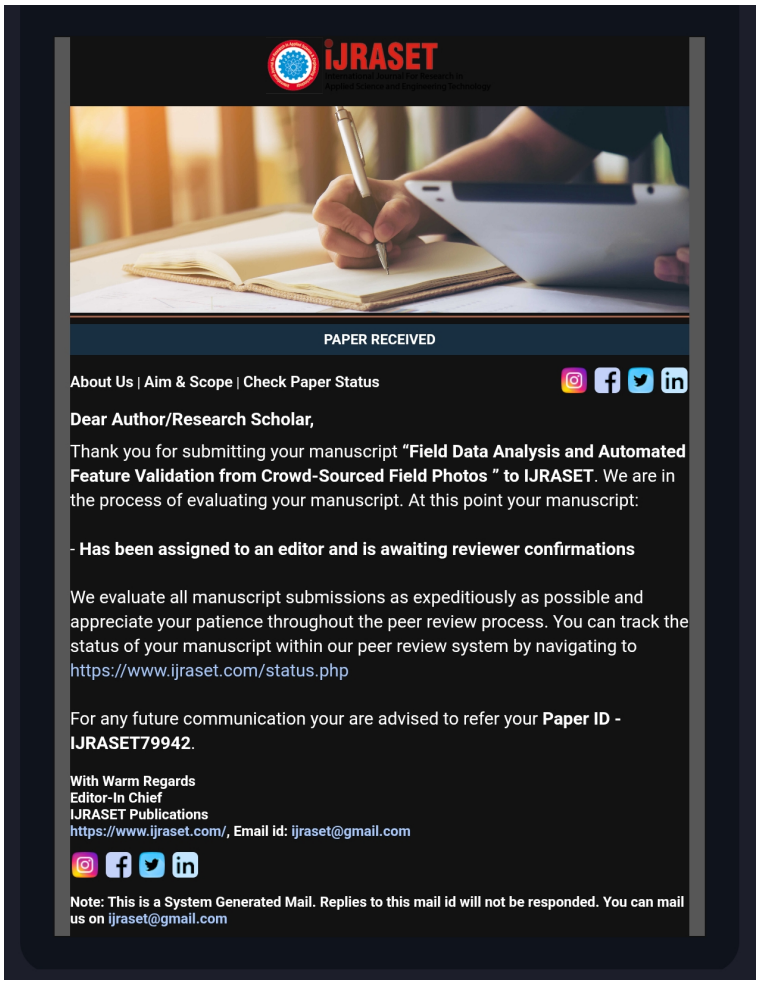


Figure 8.1: Final Project Poster

Certificate

Acceptance Email Screenshot



iJRASET

PAPER RECEIVED

About Us | Aim & Scope | Check Paper Status

Dear Author/Research Scholar,

Thank you for submitting your manuscript **"Field Data Analysis and Automated Feature Validation from Crowd-Sourced Field Photos " to IJRASET**. We are in the process of evaluating your manuscript. At this point your manuscript:

- **Has been assigned to an editor and is awaiting reviewer confirmations**

We evaluate all manuscript submissions as expeditiously as possible and appreciate your patience throughout the peer review process. You can track the status of your manuscript within our peer review system by navigating to https://www.ijraset.com/status.php

For any future communication your are advised to refer your **Paper ID - IJRASET79942**.

With Warm Regards
Editor-In Chief
IJRASET Publications
https://www.ijraset.com/, Email id: ijraset@gmail.com

Note: This is a System Generated Mail. Replies to this mail id will not be responded. You can mail us on ijraset@gmail.com

Figure 8.2: Acceptance Email for Research Paper

Plag Report

The project must be original and functional. The similarity index of the project report (including references) must be less than 20% overall and not more than 5% from any single source. The project report must contain 0% AI-generated content.

You need to check plagiarism from chapter no-1 to chapter no-9 and references of report.

GitHub Link

Link:

”https://github.com/Prachi234/Automated-Feature-Validation”